

基于DeepSeek-V4图模式加速实践

分享嘉宾：周佳龙
2026.06.27 中国·上海



 背景

 npugraph_ex图模式介绍

 总结展望



背景

DeepSeek-V4长序列低时延推理

大模型参数量

DeepSeek-V4 Pro 1.6T(49B激活)

DeepSeek-V4 Flash 284B(13B激活)

极端上下文长度

最长支持 1M序列

低时延推理部署诉求

Decode 单 token 输入 shape 稳定

存在算子CPU到NPU频繁下发瓶颈

→ **npugraph_ex图模式加速理想场景**

DeepSeek-V4 开箱性能

TPOT < 10ms

DeepSeek-V4 Flash · 950DT 16 卡 · 128K 序列

4388 TPS@30ms

DeepSeek-V4 Flash · Atlas-A3 Pod 64 卡 · 8K 序列

平台 / 规格	Chips	MTP	DataType	Seq	TPOT(ms)	Tokens/p/s
950DT Flash	16	3	MXFP8-MXFP4	8192	8.78	1821.75
950DT Flash	16	3	MXFP8-MXFP4	131072	9.66	1655.64
950DT Flash	16	3	MXFP8-MXFP4	1048576	9.46	422.88
A3 Pod Flash	64	1	W8A8C16	8192	29.17	4388

平台/规格: **A3 Pod Flash** Seq:8k Chips:16下, 开启单算子 TPOT: **283.84 ms**, 开启npugraph_ex TPOT: **30.94ms**

DeepSeek-V4真实接入代码：一行接入npugraph_ex图模式，低成本，低时延

```
# cann-recipes-infer/models/deepseek-v4/runner_deepseek.py
if self.execute_mode == "npugraph_ex":
    compile_options = {
        "frozen_parameter": True,
        "static_kernel_compile": self.enable_static_kernel,
    }
    self.model.decode = torch.compile(
        self.model.decode,
        dynamic=False, fullgraph=True,
        backend="npugraph_ex",
        options=compile_options)
```

decode 图编译

runner_deepseek.py::graph_compile

顺带打开开关

options 里启用 frozen_parameter /
static_kernel_compile

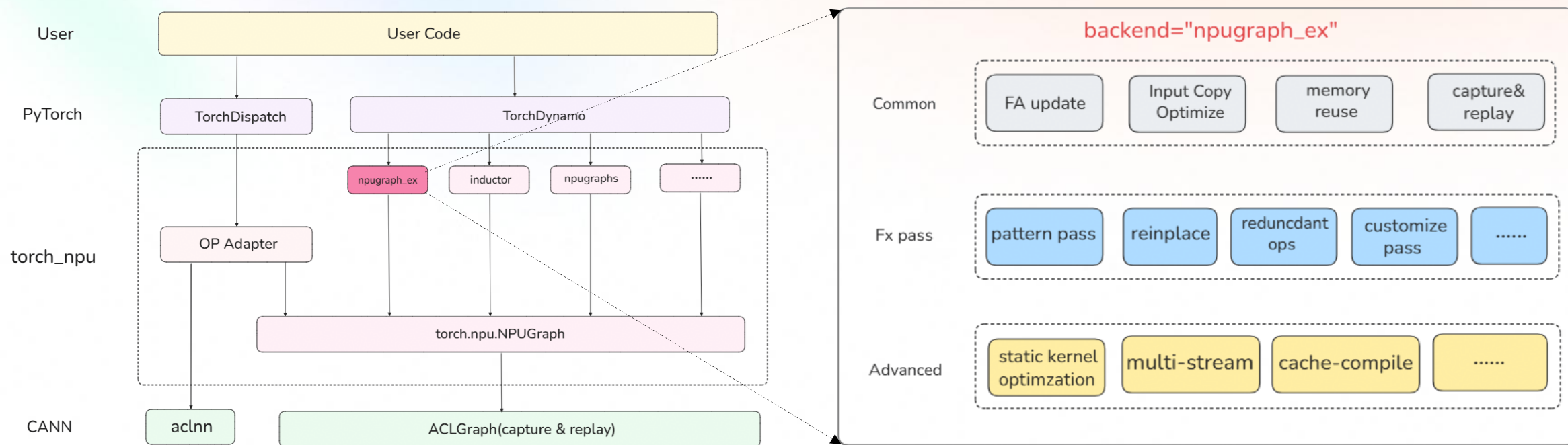
跨模型复用

同仓 GLM-5 复用完全相同写法 → 直接套用



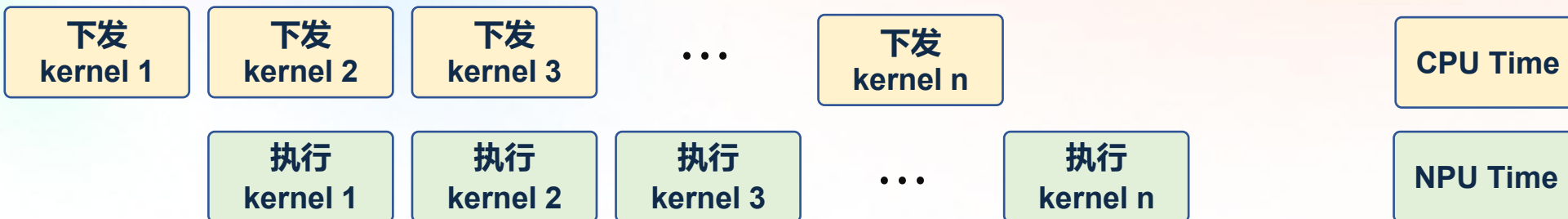
npugraph_ex图模式介绍

npugraph_ex图模式总览



npugraph_ex图模式做了什么：从逐算子下发到整图执行

Eager 执行：逐算子下发，**每个算子都要 CPU→NPU 逐个下发**



整图执行：Capture&Replay，**一次捕获、多次执行 —— 把调度从 CPU 搬到 NPU**



npugraph_ex图模式关键特性：自定义算子零额外成本入图

```
# cann-recipes-infer\models\deepseek-v4-flash-tilelang-and-inductorAF\model.py
tllib = torch.library.Library("tl", "FRAGMENT")
tllib.define("hc_split_sinkhorn(Tensor mixes, Tensor hc_scale, Tensor hc_base, int
hc_mult=4, \
    int sinkhorn_iters = 20, float eps=1e-6) -> (Tensor, Tensor, Tensor)")
```

```
@torch.library.impl(tllib, "hc_split_sinkhorn", "Meta")
def hc_split_sinkhorn_meta(mixes, hc_scale, hc_base, hc_mult=4, sinkhorn_iters=20,
eps=1e-6):
    b, s, _ = mixes.size()
    pre = mixes.new_empty(b, s, hc_mult)
    post = mixes.new_empty(b, s, hc_mult)
    comb = mixes.new_empty(b, s, hc_mult, hc_mult)
    return pre, post, comb
```

```
@torch.library.impl(tllib, "hc_split_sinkhorn", "NPU")
def hc_split_sinkhorn_npu_impl(mixes, hc_scale, hc_base, hc_mult=4,
sinkhorn_iters=20, eps=1e-6):
    return tl_hc_split_sinkhorn(mixes, hc_scale, hc_base, hc_mult, sinkhorn_iters,
eps)
```

复用torch.compile必需的算子Meta推导

给出输入到输出的 shape/dtype关系即可

自定义原地算子无需注册Functionalize

Torch.compile自动完成函数化

Npugraph_ex: 自定义原地算子(函数化) >>> 自定义原地算子

自定义算子零额外成本入图：复用必需的算子Meta推导

npugraph_ex图模式关键特性：根据输入shape信息，算子**预先静态编译**

```
torch.compile(  
    model,  
    backend="npugraph_ex",  
    options={  
        "static_kernel_compile": True,  
    },  
    fullgraph=True,  
    dynamic=False,  
)
```

存储利用率高

编译期已知所有 Tensor 大小

针对性优化

按实际 shape 优化

减少 scalar 打断

编译期完成标量计算

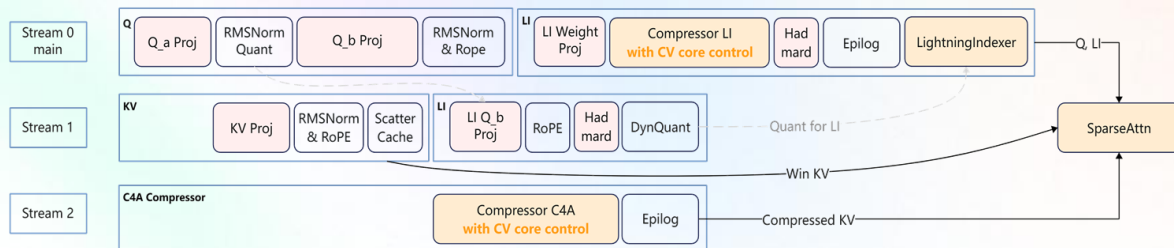
算子编译工具op_compiler

不会额外插同步，避免并行指令被
串行化

固定 shape 预编译：一个开关换来运行期更短路径

npugraph_ex图模式关键特性：无缝接入多流并行 + 限核

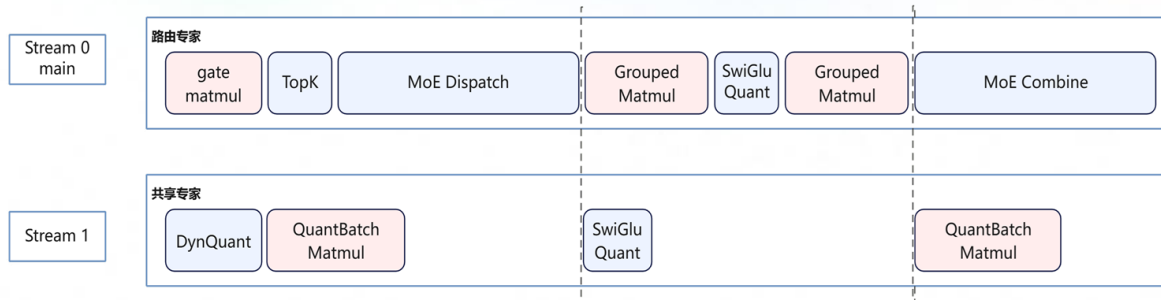
C4A多流



两类并行：计算与计算（数据依赖） / 计算与通信

MoE多流

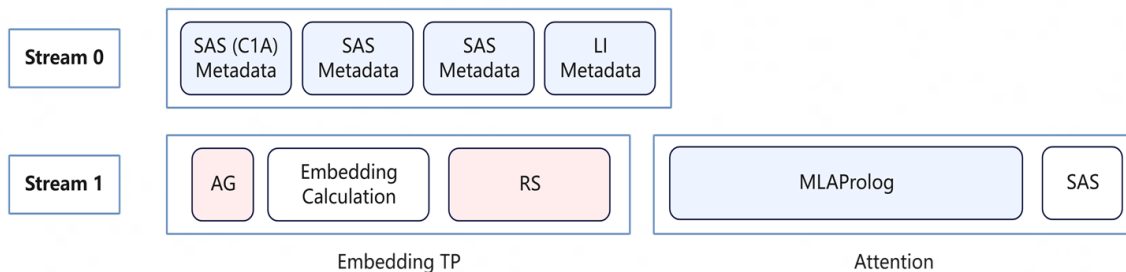
DeepSeek-V4 Attention 多流并行



限核：Stream 级配 AI / Vector Core，防抢核

MoE共享路由多流并行

Prepare Metadata



AICPU Scheduler算子多流并行

无缝接入：不需要通过配置项options额外配置

options：开关即用

特性	开启方式 / 键
FX 图优化 / In-place Pass(默认开启)	options={"inplace_pass": True, "input_inplace_pass": True}
算子融合 Pass（默认开启）	options={"pattern_fusion_pass": True}
Force eager功能	options={"force_eager": True}
静态 Kernel 编译	options = {"static_kernel_compile": True, "disable_static_kernel_compile_cache": False}
固定权重输入地址	options={"frozen_parameter": True}
冗余算子消除(默认开启)	options={"remove_noop_ops": True}
cat算子消除(默认开启)	options={"remove_cat_ops": True}
Force recapture功能	options={"force_recapture": True}
重捕获次数限制	options={"capture_limit": 64}

绝大多数能力通过`torch.compile(model, backend="npugraph_ex", options={})`

开关即可启用，无需改写网络



总结与展望

Conclusion & Future 一行接入，开箱即用

一行代码

```
torch.compile(...,  
backend="npugraph_ex",  
fullgraph=True,  
dynamic=False)
```

配置开关

多流 / 缓存 /
静态 Kernel / 限核
布尔开关

开箱特性

融合 / 内存复用 /
编译缓存 / 多流 / 限核
options 即用;
自定义算子仅需 Meta

Future: SuperKernel 进一步降算子启动与调度间隙

扫码添加CANN
TorchAir助手，加入社区
建设贡献以及SIG



扫码加入CANN开源社
社区-基础图框架微信群



Cannbot Torch-compile
插件 快速了解图模式，提
高开发效率



Npugraph_ex开发者文档，
获取技术细节与详细指导



THANKS

